

IMPLEMENTATION_PLAN

BobaLink Browser Extension — Master Implementation Plan

Revision: 1 **Last modified:** 2026-06-10T05:12:40Z **Status:** Planning complete — ready for phased, subagent-driven implementation **Authority:** This plan is derived exhaustively from docs/research/browser_extension/ (205 files, ~50k lines) via the analysis files in _analysis/01–06.md and the planning artifacts in _plan/A–F.md. Per §11.4.118, the traceability matrix (_plan/C-traceability-matrix.md, 245 items) proves nothing is skipped.

0. Hard constraints (non-negotiable — read first)

1. **NO CI/CD — EVER.** No .github/workflows/, no .gitlab-ci.yml, no Jenkins, no pipelines, no git hooks added for this subproject. The reference ships ci.yml/release.yml — they are **DROPPED**; their build/test/lint *commands* are extracted into a manual extension/ci-ext.sh shelled from the repo's ./ci.sh. (Operator-reinforced 2026-06-10; Boba Hard Stop.)
2. **Anti-bluff (§11.4 / §11.4.107).** Every test asserts a user-observable outcome and must fail against a no-op stub. The reference suite's 10 known bluffs (_plan/D) are remediated, not adopted.
3. **§11.4.10 credentials.** The extension stores **no decryptable secret by default** (delegate-by-default, _plan/E). The hard-coded "bobalink-extension" passphrase + empty-passphrase decrypt are **forbidden**. No real credential in any test/fixture/doc.
4. **§11.4.113 no force-push; §11.4.71 fetch-first; SSH remotes only.**
5. **100% multi-type test coverage (§11.4.27/§11.4.85):** unit, integration, e2e, security, load, scaling, chaos, stress, performance, benchmark, UI, UX, Challenges, HelixQA banks

- + autonomous QA sessions — for every feature/flow/use-case/edge-case.
 - 6. **§11.4.125/§11.4.134/§11.4.142 mandatory code-review** of every change before it is “complete,” iterate-until-GO.
 - 7. **Real backend = Boba merge-service on 7187** (`_plan/A`), not direct qBittorrent, not port 8080.
-

1. Product definition

BobaLink — a cross-browser **Manifest V3 WebExtension** (WXT + TypeScript strict, Vitest+Playwright) that: - detects **magnet links** + **.torrent files** on any page (MutationObserver, SPA-aware) and across **tab groups**; - parses them (bencode / magnet BEP-9 / .torrent → SHA-1 infohash), dedups by infohash; - forwards them to **Boba’s merge-service (:7187)**, which adds them to qBittorrent server-side; - provides popup + options UI, offline FIFO queue, badge/notifications, context menu, keyboard shortcuts, i18n (8 locales), WCAG 2.1 AA, dark/light themes; - targets Chrome ≥109, Firefox ≥109 (MV3 event pages via WXT), Opera, Edge, Yandex (Safari deferred).

Supported browsers / data model / sequences / state machines: see `_analysis/03-diagrams.md` (6 ER entities, 2 end-to-end sequences, offline-queue + lifecycle state machines, security architecture, compatibility matrix).

2. Repository placement & naming (proposed — operator-objectable)

- **Location:** new top-level extension/ directory (sibling to frontend/, download-proxy/). Snake_case per §11.4.29 where it doesn’t break tooling; the WXT/TS conventions inside extension/src/ follow the framework.
 - **Name:** BobaLink (from the materials). Objectable.
 - **Build artifact:** extension/.output/chrome-mv3-prod/ (+ firefox), zipped for manual store submission (no auto-submit).
-

3. Backend additions required in Boba (Python merge-service)

From `_plan/A` — the extension needs capabilities Boba doesn’t yet expose. These are **Phase 4** tasks (real, tested, anti-bluff): - **BE-1 — CORS for extension origins:** add `chrome-extension://*` + `moz-extension://*` to `ALLOWED_ORIGINS` (or document the host-permission

background-fetch path that avoids CORS). File: download-proxy/src/api/__init__.py CORS middleware. - **BE-2 — raw .torrent upload:** new POST /api/v1/download/upload (multipart bytes → qBittorrent /api/v2/torrents/add multipart). Boba currently accepts URLs only. File: download-proxy/src/api/routes.py. - **BE-3 — Go-profile parity (optional):** qBitTorrent-go/internal/api/download.go is a stub that echoes "added" without contacting qBittorrent — either implement it or document that the extension targets the Python backend only.

Each backend addition follows full TDD + four-layer coverage + its own code-review GO.

4. Phased implementation plan

Each phase lists **Tasks** → **fine-grained Sub-tasks**, the **traceability IDs** it satisfies (see `_plan/C`), the **adopt/refactor/rewrite/net-new** disposition (`_plan/F`), and the **test types** that gate it (`_plan/D`). Every phase ends with a code-review GO (§11.4.125) before merge. Phases are subagent-driven (§11.4.70), parallelized where file-scopes are disjoint (§11.4.103).

Phase 1 — Foundation & scaffolding (*Disposition: REFACTOR configs / ADOPT types & shared libs*)

Covers: build/config, data model, shared libs, constants, types. Traceability: FND-, ER-, NFR-maintainability. - **T1.1 Scaffold WXT project** — extension/ with package.json (SSH repo URL, node ≥20, ESM), wxt.config.ts (MV3, minimum_chrome_version 109), tsconfig.json (strict, zero any), .eslintrc/.prettierrc. **NO .github/.** - Sub: port the npm scripts (dev/build/zip/compile/lint/test) — drop CI-only scripts. - Sub: WXT auto-icons → rasterize the missing icon PNGs from assets/icon.svg (net-new). - **T1.2 Manifest/permissions/CSP** — least-privilege per `_plan/E`: storage, alarms, notifications, activeTab, contextMenus; host_permissions localhost-only (:7187 + optional remote); MV3 CSP object form; **no <all_urls>/tabs/cookies**. Re-target all :8080 → :7187. - **T1.3 Types & data model** — ADOPT types/torrent.ts; de-dup AuthMethod/ServerConfig drift; reconcile the SQL schema (9 tables) vs chrome.storage JSON — **decision: chrome.storage.local/session is the source of truth for v1** (sql.js deferred unless needed); document the ER model as the logical schema. - **T1.4 Shared libs** — ADOPT crypto.ts (sound primitive), errors.ts, events.ts, logger.ts, storage.ts, utils.ts; fix the RateLimitError/ServerError name mismatch; constants port (ports/URLs/timeouts/

STORAGE_KEYS). - **Tests:** unit (shared libs, constants, type guards) + the foundation Challenge (artifact-open §11.4.38). Vitest configured with UI/entrypoints **in** coverage.

Phase 2 — Core detection & parsing engine (ADOPT parsers/scanners + REFACTOR)

Covers: FR-001..FR-010 (detection/parsing/infohash/dedup), Dim06/Dim07/Dim08, EDGE-*. - **T2.1 Parsers** — ADOPT bencode.ts, magnet.ts; **REFACTOR torrent-file.ts** (wire the dead .torrent parser; SHA-1 infohash = SHA-1(bencode(info)) via WebCrypto; private-passkey sanitization). - **T2.2 Scanners** — ADOPT link-scanner.ts/text-scanner.ts/content/scanner.ts; **REFACTOR scanner/base.ts** (remove Date.now()-salted unstable IDs → stable infohash-based IDs); **merge the two divergent site-db.ts selector tables** into one; MutationObserver 500ms debounce; Shadow-DOM recursion; iframe all_frames; 16ms frame budget + cooperative yielding. - **T2.3 Dedup** — by lowercase infohash (FR-007). - **Tests:** unit (every bencode type, every magnet param xt/dn/tr/xl/ws/x.pe, hex+base32 infohash, malformed rejection, infohash determinism) — **remediate** the bencode dead-assert; performance (≤5ms/link on 1000-link page); load (10k-link page, no frame >16ms); stress (§11.4.85 boundary inputs); benchmark.

Phase 3 — Extension shell (content / background / popup / options) (REFACTOR)

Covers: FR-011..FR-018 (UI, context menu, shortcuts, badge), message protocol, lifecycle/state machines. - **T3.1 Content script** — REFACTOR content/index.ts (unify the get-detected message contract); highlight UI; message passing to background. - **T3.2 Background SW** — REFACTOR background/index.ts (message router; context menus ×4; chrome.commands ×3-4; chrome.alarms keep-alive ~20s + health 5min; badge/notifications); **remove the empty-passphrase decrypt** (→ Phase 7 secure model). Offline-queue state machine (_analysis/03 #09): Idle→Pending→{Sending|Queued|Retry}→{Completed|Failed}, backoff 5s→5min, ≤5 retries, max 1000, Dead-Letter. - **T3.3 Popup** — REFACTOR popup/popup.ts (detected list, Send, Send-All, status icon); every control → ExtensionConfig field. - **T3.4 Options** — REFACTOR options/options.ts (7 tabs: Server/Download/Queue/Notifications/Detection/UI/Security); **remove the fixed "bobalink-extension" passphrase**. - **Tests:** e2e (real extension loaded via persistent-context --load-extension, real id resolved from SW target — fixes the test-id bluff); UI/UX (WCAG 2.1 AA, keyboard nav, ARIA); integration (message protocol).

Phase 4 — Boba backend integration (*NET-NEW + the BE-1/2/3 backend additions*)

Covers: FR-019..FR-023 (send, queue, health, auth), Dim01/Dim09 (Dim01 authoritative). - **T4.1 Real Boba client** — NET-NEW api/boba-client.ts: POST /api/v1/download (magnet + URL), GET /health, GET /api/v1/auth/status; retry (exp backoff + jitter, 3×, 1s→30s); TokenBucket rate-limit (FR-025); AbortController timeout; auto-discovery of :7187 (+ 7186/7189 fallback probes). - **T4.2 Offline queue auth + persistence** — REFACTOR api/queue.ts (inject the real send path; the reference queue never authenticated); durability across SW restart (100%, NFR). - **T4.3 Backend additions (Boba side)** — BE-1 CORS, BE-2 /api/v1/download/upload, BE-3 Go stub decision (§3 above). TDD on the Python/Go backend. - **Tests:** integration against **live** 7187 (require_backend(7187) → SKIP-with-reason if down, never fail-open §11.4.3/§11.4.68); security (no creds sent to qBittorrent from the extension); chaos (backend drops mid-send → queue recovers); the end-to-end Challenge (detect→send→**torrent appears in qBittorrent**).

Phase 5 — Tab-group batch (*NET-NEW — does not exist in the reference*)

Covers: FR-024, Dim05, sequence #05. - **T5.1** chrome.tabGroups.query → chrome.tabs.query{groupId} → chrome.scripting.executeScript per tab → scan → flatten + dedup → batch send (urls=magnet1\nmagnet2...). - **T5.2** Context-menu “Send tab group” + popup “Send All in group”; collapsed-group handling; Firefox-incompatible → graceful degrade. - **Tests:** e2e (real multi-tab group), integration (batch dedup), UX (progress notification), Challenge (tab-group batch → N torrents in qBittorrent).

Phase 6 — UI/UX, i18n, accessibility, themes (*REFACTOR/ADOPT*)

Covers: FR-016/FR-017, Dim10, NFR-a11y. - **T6.1** i18n (_locales — 8 locales; default_locale); **T6.2** dark/light via prefers-color-scheme; **T6.3** WCAG 2.1 AA audit (contrast, focus, ARIA, keyboard); **T6.4** notification templates (no Base64 icons; Mac limits); badge ≤4 chars. - **Tests:** UX (a11y automated + heuristic), UI (every control state), i18n (locale switch renders).

Phase 7 — Security & credentials (*NET-NEW* key source + *REFACTOR* call-sites; crypto module *ADOPTED*)

Covers: SEC-*, Dim11, diagram #10, §11.4.10. Full design: `_plan/E`. - **T7.1 Credential model** — delegate-by-default (extension holds **no decryptable secret** in the localhost-proxy deployment); residual remote-Boba token → user-supplied session passphrase (PBKDF2, `chrome.storage.session` only, never persisted). Remove both broken call-sites. - **T7.2 Controls** — least-privilege manifest, MV3 CSP, HTTPS-off-localhost + cert verify, magnet/dn XSS sanitization, `passkey`→`***PASKEY***` log redaction, no-clear-text-credential-logging, `sender.id`/`sender.url` validation, GDPR “Clear All Data.” - **Tests (anti-bluff, each RED-on-broken first §11.4.115, permanent guards §11.4.135):** T2 wrong-key fails (the “boba-link-extension” + “” decrypts both THROW), T3 no embedded key (`grep src + bundle` → zero hits), T4 no secret in logs, T1 ciphertext-at-rest, T8 delegation proof (no qBt/tracker creds in extension state); security/pen-test suite; §11.4.10.A pre-store leak audit + pre-commit grep gate.

Phase 8 — Testing to 100% (all types) + Challenges + HelixQA (*NET-NEW*)

Covers: the 13 mandated test types, every feature. Full matrix: `_plan/D`. - **T8.1 Remediate the 10 reference bluffs** (B1-B10): inline-crypto→import real module; Jest→Vitest; `expect(true)` no-ops→real assertions; non-functional e2e→real-extension fixture + global-setup/teardown; coverage includes UI/entrypoints. - **T8.2 Fill missing test types** — integration, security, load (10k links + API flood vs rate-limiter), scaling, chaos (backend death, storage-quota, SW termination), stress (§11.4.85 ≥100 iters/≥10 concurrent, p50/p95/p99), performance + benchmark (NFR thresholds), UI, UX. - **T8.3 Challenges** — 8 challenges/scripts/`boba_ext_*.sh` driving the **real built extension** end-to-end (detect→send→qBittorrent row; offline-queue durability; cred-at-rest; auto-discover; tab-group batch; artifact-open §11.4.38; cred-leak grep). No false success. - **T8.4 HelixQA banks** — submodules/`helixqa/banks/boba-boba-link.yaml` (http: style, backend endpoints, CI-gated) + `boba-boba-link-ui.yaml` (Playwright, extension UI). Resolve the helixqa build-blocker (add the 6 sibling submodules **or** consume bank-YAML only). Autonomous QA session, re-runnable N× (§11.4.98/§11.4.50), live JSONL verdict stream (§11.4.116). - **T8.5** Coverage ledger → 100% feature × test-type; ratchet thresholds; every green proves a user-observable outcome.

Phase 9 — Build, packaging & distribution (manual — NO CI/CD)

Covers: BLD-*, Dim12, diagram #11. - **T9.1** extension/ci-ext.sh (manual gate: lint → format:check → tsc --noEmit → vitest -coverage → playwright → wxt build → zip + manifest-validate + bundle-size ≤350KB), shelled from ./ci.sh. **NO GitHub Actions / GitLab pipelines.** - **T9.2** Per-store packaging (Chrome/Firefox-gecko.id/Opera/Edge/Yandex) — **manual** zips; document submission steps; no auto-submit, no store secrets in git. - **T9.3** User + developer + installation + API docs (port the 5 reference guides, corrected for the real 7187 contract), with §11.4.65 html/pdf siblings.

5. Test/Challenge/HelixQA coverage strategy (summary — full in _plan/D)

- **Runner:** Vitest + Playwright (matches Boba frontend/; one coverage tool @vitest/coverage-v8).
- **E2E:** wxt build → launchPersistentContext + --load-extension → real id from SW target.
- **Live integration:** require_backend(7187) SKIP-with-reason if down (never fail-open).
- **Coverage:** UI/entrypoints **in scope**; ratchet ≥90%→100; anti-bluff is the real gate.
- **Per-feature × per-test-type matrix:** 14 types × 13 modules; N/A cells justified (DDoS N/A for a client parser, but Load IS in-scope for 10k-link pages + rate-limiter flood).

6. Adopt/Refactor/Rewrite map (summary — full in _plan/F)

~80% reusable. ADOPT: parsers (bencode/magnet), scanners, shared libs incl. the crypto **module** (sound), types, i18n. REFACTOR: api targets (8080→7187), torrent-file wiring, scanner IDs/site-db, the two credential call-sites. REWRITE: the test suite's bluffs + configs. NET-NEW: real Boba client, tab-groups, secure key source, the 13 missing test types, icon PNGs. DROP: both .github/workflows/*.yml.

7. Open operator decisions

1. **Subproject location** — extension/ at repo root (proposed). Object if you want it elsewhere.

2. **Backend additions** — OK to add BE-1 (CORS) + BE-2 (/api/v1/download/upload) to the Python merge-service? (Required for .torrent-bytes upload + extension-origin calls.)
3. **HelixQA build-blocker** — add the 6 missing sibling submodules (doc_processor, llm_orchestrator, llm_provider, llms_verifier, security, vision_engine) so bin/helixqa builds in-repo, **or** consume bank-YAML only and run helixqa from a standalone checkout?
4. **Session scope** — proceed to Phase 1 scaffolding now (autonomous), or hold for your review of this plan first?
5. **Product name** BobaLink — keep?

8. Traceability & provenance

- Full requirements matrix (245 items, 240 v1 / 5 v2): _plan/C-traceability-matrix.md — every FR/NFR/Dim/ER/edge-case placed; none untraceable.
- Source extractions (every file read, manifests prove coverage): _analysis/01-06.md.
- Planning evidence: _plan/A (backend contract), B (HelixQA), C (traceability), D (tests), E (security), F (adopt/rewrite).
- v2-deferred (documented seams, not dropped): multi-client gateway, search_cache, Boba search/SSE, side-panel, qBt API-key auth.